

Trabajo Práctico: Consultas SQL

Materia: Base de Datos — Unidad 2 — Resolución de Enunciados

A) Consultas simples (una sola tabla)

1) Obtener una lista con los nombres de las distintas oficinas de la empresa.

```
SELECT DISTINCT descripcion
FROM Oficinas;
```

2) Listar cada producto indicando descripción, precio de costo y precio con IVA (21%).

```
SELECT descripcion,
       precio          AS precio_costo,
       precio * 1.21   AS precio_con_iva
FROM Productos;
```

3) Listar cada empleado con apellido, nombre, fecha de cumpleaños y edad actual.

```
SELECT apellido,
       nombre,
       fecha_nacimiento          AS fecha_cumpleanos,
       TIMESTAMPDIFF(YEAR, fecha_nacimiento, CURDATE()) AS edad
FROM Empleados;
```

4) Listar todos los empleados que tienen un jefe asignado.

```
SELECT *
FROM Empleados
WHERE cod_jefe IS NOT NULL;
```

5) Listar los empleados de nombre "María" ordenados por apellido.

```
SELECT *
FROM Empleados
WHERE nombre = 'María'
ORDER BY apellido ASC;
```

6) Listar los clientes cuya razón social comience con "L", ordenado por código de cliente.

```
SELECT *
FROM Clientes
WHERE razón_social LIKE 'L%'
ORDER BY cod_cliente ASC;
```

7) Listar toda la información de los pedidos de Marzo, ordenado por fecha de pedido.

```
SELECT *  
FROM Pedidos  
WHERE MONTH(fecha_pedido) = 3  
ORDER BY fecha_pedido ASC;
```

8) Listar las oficinas que no tienen director asignado.

```
SELECT *  
FROM Oficinas  
WHERE codigo_director IS NULL;
```

9) Listar los 4 productos de menor precio de costo.

```
SELECT *  
FROM Productos  
ORDER BY precio ASC  
LIMIT 4;
```

10) Listar los códigos de los 3 empleados con mayor cuota.

```
SELECT cod_empleado  
FROM Datos_contratos  
ORDER BY cuota DESC  
LIMIT 3;
```

Consultas multitablas

1) De cada producto listar descripción, razón social del fabricante y stock, ordenado por razón social y descripción.

```
SELECT p.descripcion,  
       f.razón_social,  
       p.cantidad_stock  
FROM Productos p  
INNER JOIN Fabricantes f ON p.cod_fabricante = f.cod_fabricante  
ORDER BY f.razón_social ASC, p.descripcion ASC;
```

2) De cada pedido listar código, fecha, apellido del empleado y razón social del cliente.

```
SELECT p.cod_pedido,  
       p.fecha_pedido,  
       e.apellido,  
       c.razón_social
```

```

FROM Pedidos p
INNER JOIN Empleados e ON p.cod_empleado = e.cod_empleado
INNER JOIN Clientes c ON p.cod_cliente = c.cod_cliente;

```

3) Listar por cada empleado: apellido, cuota asignada y oficina, ordenado de forma descendente por cuota.

```

SELECT e.apellido,
       dc.cuota,
       o.descripcion AS oficina
FROM Empleados e
INNER JOIN Datos_contratos dc ON e.cod_empleado = dc.cod_empleado
LEFT JOIN Oficinas o ON e.cod_oficina = o.cod_oficina
ORDER BY dc.cuota DESC;

```

4) Listar sin repetir la razón social de los clientes que hicieron pedidos en Abril.

```

SELECT DISTINCT c.razón_social
FROM Clientes c
INNER JOIN Pedidos p ON c.cod_cliente = p.cod_cliente
WHERE MONTH(p.fecha_pedido) = 4;

```

5) Listar sin repetir los productos que fueron pedidos en Marzo.

```

SELECT DISTINCT pr.cod_producto,
               pr.descripcion
FROM Productos pr
INNER JOIN Detalle_pedidos dp ON pr.cod_producto = dp.cod_producto
INNER JOIN Pedidos p ON dp.cod_pedido = p.cod_pedido
WHERE MONTH(p.fecha_pedido) = 3;

```

6) Listar empleados contratados hace más de 10 años, ordenado por años de antigüedad de forma descendente.

```

SELECT e.cod_empleado,
       e.apellido,
       e.nombre,
       TIMESTAMPDIFF(YEAR, dc.fecha_contrato, CURDATE()) AS cantidad_anios
FROM Empleados e
INNER JOIN Datos_contratos dc ON e.cod_empleado = dc.cod_empleado
WHERE TIMESTAMPDIFF(YEAR, dc.fecha_contrato, CURDATE()) > 10
ORDER BY cantidad_anios DESC;

```

7) Obtener una lista de los clientes mayoristas ordenada por razón social.

```

SELECT c.cod_cliente,
       c.razón_social

```

```

FROM Clientes c
INNER JOIN Listas l ON c.cod_lista = l.cod_lista
WHERE l.descripcion LIKE '%Mayorista%'
ORDER BY c.razón_social ASC;

```

8) Listar sin repetir qué productos compró cada cliente, ordenado por razón social y descripción.

```

SELECT DISTINCT c.razón_social,
                pr.descripcion AS descripcion_producto
FROM Clientes c
INNER JOIN Pedidos p ON c.cod_cliente = p.cod_cliente
INNER JOIN Detalle_pedidos dp ON p.cod_pedido = dp.cod_pedido
INNER JOIN Productos pr ON dp.cod_producto = pr.cod_producto
ORDER BY c.razón_social ASC, pr.descripcion ASC;

```

9) Listar productos con stock por debajo del punto de reposición, indicando cantidad a comprar y fabricante.

```

SELECT p.descripcion,
       (p.punto_reposición - p.cantidad_stock) AS cantidad_a_comprar,
       f.razón_social
FROM Productos p
INNER JOIN Fabricantes f ON p.cod_fabricante = f.cod_fabricante
WHERE p.cantidad_stock < p.punto_reposición
ORDER BY f.razón_social ASC, p.descripcion ASC;

```

10) Listar empleados cuya cuota es menor a 50.000 o mayor a 100.000.

```

SELECT e.cod_empleado,
       e.apellido,
       dc.cuota
FROM Empleados e
INNER JOIN Datos_contratos dc ON e.cod_empleado = dc.cod_empleado
WHERE dc.cuota < 50000 OR dc.cuota > 100000;

```

B) Consultas sobre segundo esquema

1) Obtener la cantidad de unidades máxima.

```

SELECT MAX(cantidad) AS cantidad_maxima
FROM ITEM_VENTAS;

```

2) Obtener la cantidad total de unidades vendidas del producto c.

```

SELECT SUM(iv.cantidad) AS total_vendido

```

```

FROM ITEM_VENTAS iv
INNER JOIN PRODUCTOS p ON iv.coódigo_producto = p.código_producto
WHERE p.nombre_producto = 'c'
      OR p.código_producto = 'c';

```

3) Cantidad de unidades vendidas por producto, con descripción, ordenado de mayor a menor.

```

SELECT p.nombre_producto AS descripcion_producto,
       SUM(iv.cantidad)   AS cantidad_total
FROM PRODUCTOS p
INNER JOIN ITEM_VENTAS iv ON p.código_producto = iv.coódigo_producto
GROUP BY p.código_producto, p.nombre_producto
ORDER BY cantidad_total DESC;

```

4) Cantidad vendida por producto para los que vendieron más de 30 unidades, ordenado alfabéticamente.

```

SELECT p.nombre_producto AS descripcion_producto,
       SUM(iv.cantidad)   AS cantidad_total
FROM PRODUCTOS p
INNER JOIN ITEM_VENTAS iv ON p.código_producto = iv.coódigo_producto
GROUP BY p.código_producto, p.nombre_producto
HAVING SUM(iv.cantidad) > 30
ORDER BY p.nombre_producto ASC;

```

5) Cuántas compras realizó cada cliente (1 factura = 1 compra), ordenado de mayor a menor.

```

SELECT c.código_cliente,
       c.nombre,
       COUNT(v.número_factura) AS total_compras
FROM CLIENTES c
INNER JOIN VENTAS v ON c.código_cliente = v.código_cliente
GROUP BY c.código_cliente, c.nombre
ORDER BY total_compras DESC;

```

6) Promedio de unidades vendidas por producto para el cliente 1.

```

SELECT iv.coódigo_producto,
       AVG(iv.cantidad) AS promedio_unidades
FROM ITEM_VENTAS iv
INNER JOIN VENTAS v ON iv.número_factura = v.número_factura
WHERE v.código_cliente = 1
GROUP BY iv.coódigo_producto;

```